UNIVERSIDADE FEDERAL DO PARÁ
INSTITUTO DE CIÊNCIAS EXATAS E NATURAIS
FACULDADE DE COMPUTAÇÃO – FACOMP/UFPA

TAREFA AVALIATIVA
Estruturas de Dados

Pilhas, Árvores Binárias de Busca e Árvores AVL

Docente	Lídio
Discentes	Alice Fernanda Silva Barros João Emannuel Neves João Victor Lisboa
Repositório (códigos-fonte)	https://github.com/aliceb404/Estrutura-de-dados-II/tree/main
Valor	5,0 pontos

Conteúdo deste documento

- **Questão 1** – Pilha sequencial (PUSH/POP) com código e saída da execução.
- **Questão 2** – Permutações obtidas por uma pilha: 14 possíveis e 10 impossíveis, com as operações de cada uma.
- **Questão 3** – Árvore Binária de Busca: código completo, desenhos após cada operação e itens (a) a (f).
- **Questão 4** – Árvore AVL: rotações, desbalanceamentos, remoções e desafio adicional.

Questão 1 – Pilha sequencial (LIFO) de páginas web

O que o código faz. A `struct Stack` guarda o vetor `dados[10]` e o inteiro `topo`; `topo = -1` indica pilha vazia. `inicializarPilha` faz o *malloc* da struct e define `topo = -1`. O `push` verifica se a pilha está cheia (`topo == MAX-1`, erro −1); se não estiver, faz `topo++` e copia o endereço com `strcpy`. O `pop` verifica se a pilha está vazia (`topo == -1`, erro −2); se não estiver, mostra o elemento do topo e faz `topo--`. As duas operações são O(1).

Código	Significado
0 (OK)	operação realizada
−1 (ERRO_PILHA_CHEIA)	overflow: push com a pilha cheia
−2 (ERRO_PILHA_VAZIA)	underflow: pop com a pilha vazia

Código-fonte (itens a, b, c e d identificados nos comentários)

```
/* QUESTAO 1 - Pilha sequencial (LIFO) de 10 espacos para paginas web visitadas */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 10

typedef struct {
    char adress[30];
    int connectionType;
} PageData;

/* a) Struct da pilha: array correspondente a LIFO + int para guardar o topo */
typedef struct {
    PageData dados[MAX];
    int topo;                     /* -1 indica pilha vazia */
} Stack;

/* Codigos de retorno */
#define OK               0
#define ERRO_PILHA_CHEIA -1       /* overflow  */
#define ERRO_PILHA_VAZIA -2       /* underflow */

/* b) Aloca a pilha */
Stack* inicializarPilha() {
    Stack* pilha = (Stack*) malloc(sizeof(Stack));
    if (pilha != NULL) {
        pilha->topo = -1;
    }
    return pilha;
}

/* c) Adiciona um elemento ao topo */
int push(Stack* pilha, char* adress, int connectionType) {
    if (pilha->topo == MAX - 1) {                        /* pilha cheia */
        printf("PUSH falhou: pilha cheia\n");
        return ERRO_PILHA_CHEIA;
    }
    pilha->topo++;
    strcpy(pilha->dados[pilha->topo].adress, adress);
    pilha->dados[pilha->topo].connectionType = connectionType;
    printf("PUSH: %-26s (conexao tipo %d) | topo = %d\n", adress, connectionType, pilha->topo);
    return OK;
}

/* d) Remove e mostra o ultimo endereco da pilha */
int pop(Stack* pilha) {
    PageData removido;
    if (pilha->topo == -1) {                             /* pilha vazia */
        printf("POP falhou: pilha vazia\n");
        return ERRO_PILHA_VAZIA;
    }
    removido = pilha->dados[pilha->topo];
    pilha->topo--;
    printf("POP : %-26s (conexao tipo %d) | topo = %d\n",
           removido.adress, removido.connectionType, pilha->topo);
    return OK;
}

int main() {
    int errorCode;
    Stack* pilha;

    pilha = inicializarPilha();   // Pilha criada
    errorCode = push(pilha, "http://icen.ufpa.br", 2);
    errorCode = push(pilha, "http://www.youtube.com", 2);
    errorCode = push(pilha, "https://www.facebook.com", 1);
    errorCode = pop(pilha);
    errorCode = push(pilha, "https://drive.google.com", 1);
```

```
    errorCode = pop(pilha);
    errorCode = pop(pilha);
    errorCode = pop(pilha);

    (void) errorCode;
    free(pilha);
    return 0;
}
```

Saída da execução

```
PUSH: http://icen.ufpa.br        (conexao tipo 2) | topo = 0
PUSH: http://www.youtube.com     (conexao tipo 2) | topo = 1
PUSH: https://www.facebook.com   (conexao tipo 1) | topo = 2
POP : https://www.facebook.com   (conexao tipo 1) | topo = 1
PUSH: https://drive.google.com   (conexao tipo 1) | topo = 2
POP : https://drive.google.com   (conexao tipo 1) | topo = 1
POP : http://www.youtube.com     (conexao tipo 2) | topo = 0
POP : http://icen.ufpa.br        (conexao tipo 2) | topo = -1
```

Os valores do topo (0, 1, 2, 1, 2, 1, 0, −1) confirmam o comportamento LIFO: o último endereço inserido é sempre o primeiro a sair.

Questão 2 – Permutações de 1, 2, 3, 4 obtidas como saída de uma pilha

Ideia. Os valores entram sempre na ordem 1, 2, 3, 4. Para fazer o valor *v* sair, o programa empilha até que *v* tenha entrado (se ainda não entrou) e confere se *v* está no topo. Se não estiver, *v* ficou preso embaixo de outro elemento que ainda precisa sair, e a permutação é impossível.

Cálculos

- Total de permutações: 4! = 24.
- Permutações possíveis: número de Catalan $C_4 = (1/5)\cdot C(8,4) = 70/5 =$ **14**.
- Permutações impossíveis: 24 − 14 = **10**.

Por que as 10 são impossíveis. Todas contêm, na saída, o padrão *"maior, menor, médio"* (por exemplo 3, 1, 2), em posições não necessariamente vizinhas. Se o 3 sai antes do 1 e do 2, os dois já estavam na pilha com o 2 por cima do 1; logo o 2 teria de sair antes do 1. Já as 14 possíveis são exatamente as permutações que não contêm esse padrão.

As 14 permutações possíveis e as operações (E = Empilha, D = Desempilha)

Permutação	Sequência de operações
1 2 3 4	`E1 D1 E2 D2 E3 D3 E4 D4`
1 2 4 3	`E1 D1 E2 D2 E3 E4 D4 D3`
1 3 2 4	`E1 D1 E2 E3 D3 D2 E4 D4`
1 3 4 2	`E1 D1 E2 E3 D3 E4 D4 D2`
1 4 3 2	`E1 D1 E2 E3 E4 D4 D3 D2`
2 1 3 4	`E1 E2 D2 D1 E3 D3 E4 D4`
2 1 4 3	`E1 E2 D2 D1 E3 E4 D4 D3`
2 3 1 4	`E1 E2 D2 E3 D3 D1 E4 D4`
2 3 4 1	`E1 E2 D2 E3 D3 E4 D4 D1`
2 4 3 1	`E1 E2 D2 E3 E4 D4 D3 D1`
3 2 1 4	`E1 E2 E3 D3 D2 D1 E4 D4`
3 2 4 1	`E1 E2 E3 D3 D2 E4 D4 D1`
3 4 2 1	`E1 E2 E3 D3 E4 D4 D2 D1`
4 3 2 1	`E1 E2 E3 E4 D4 D3 D2 D1`

A permutação 2 3 1 4 do enunciado corresponde a `E1 E2 D2 E3 D3 D1 E4 D4`, igual à do exemplo.

As 10 permutações impossíveis

Permutação	Motivo
1 4 2 3	Para sair o 2, o topo é o 3, que ainda precisa sair depois: o 2 está preso abaixo do 3.
2 4 1 3	Para sair o 1, o topo é o 3, que ainda precisa sair depois: o 1 está preso abaixo do 3.
3 1 2 4	Para sair o 1, o topo é o 2, que ainda precisa sair depois: o 1 está preso abaixo do 2.
3 1 4 2	Para sair o 1, o topo é o 2, que ainda precisa sair depois: o 1 está preso abaixo do 2.
3 4 1 2	Para sair o 1, o topo é o 2, que ainda precisa sair depois: o 1 está preso abaixo do 2.
4 1 2 3	Para sair o 1, o topo é o 3, que ainda precisa sair depois: o 1 está preso abaixo do 3.
4 1 3 2	Para sair o 1, o topo é o 3, que ainda precisa sair depois: o 1 está preso abaixo do 3.
4 2 1 3	Para sair o 2, o topo é o 3, que ainda precisa sair depois: o 2 está preso abaixo do 3.
4 2 3 1	Para sair o 2, o topo é o 3, que ainda precisa sair depois: o 2 está preso abaixo do 3.
4 3 1 2	Para sair o 1, o topo é o 2, que ainda precisa sair depois: o 1 está preso abaixo do 2.

Código-fonte

```
/* QUESTAO 2 - Permutacoes de 1,2,3,4 obtidas como saida de uma pilha */
#include <stdio.h>
#include <string.h>

#define N 4

/* Simula a pilha para obter a permutacao perm[].
 * Retorna 1 se possivel (ops recebe as operacoes: E = Empilha, D = Desempilha)
 * e 0 se impossivel. */
int simula(int perm[], char *ops) {
    int pilha[N], topo = -1, proximo = 1, i;
    char tmp[16];

    ops[0] = '\0';
    for (i = 0; i < N; i++) {
        while (proximo <= perm[i]) {              /* empilha ate perm[i] entrar */
            pilha[++topo] = proximo;
            sprintf(tmp, "E%d ", proximo++);
            strcat(ops, tmp);
        }
        if (topo < 0 || pilha[topo] != perm[i]) {  /* perm[i] ficou preso embaixo */
            return 0;
        }
        topo--;
        sprintf(tmp, "D%d ", perm[i]);
        strcat(ops, tmp);
    }
    return 1;
}

int main() {
    int a, b, c, d, total = 0, possiveis = 0;
    char ops[64];

    for (a = 1; a <= N; a++)
    for (b = 1; b <= N; b++)
    for (c = 1; c <= N; c++)
    for (d = 1; d <= N; d++) {
        int perm[N] = {a, b, c, d};
        if (a == b || a == c || a == d || b == c || b == d || c == d) continue;

        total++;
        if (simula(perm, ops)) {
            possiveis++;
            printf("%2d) %d %d %d %d  POSSIVEL    %s\n", total, a, b, c, d, ops);
        } else {
            printf("%2d) %d %d %d %d  IMPOSSIVEL\n", total, a, b, c, d);
        }
    }
    printf("\n%d possiveis e %d impossiveis (de %d)\n", possiveis, total - possiveis, total);
    return 0;
}
```

Saída da execução

```
 1) 1 2 3 4  POSSIVEL    E1 D1 E2 D2 E3 D3 E4 D4
 2) 1 2 4 3  POSSIVEL    E1 D1 E2 D2 E3 E4 D4 D3
 3) 1 3 2 4  POSSIVEL    E1 D1 E2 E3 D3 D2 E4 D4
 4) 1 3 4 2  POSSIVEL    E1 D1 E2 E3 D3 E4 D4 D2
 5) 1 4 2 3  IMPOSSIVEL
 6) 1 4 3 2  POSSIVEL    E1 D1 E2 E3 E4 D4 D3 D2
 7) 2 1 3 4  POSSIVEL    E1 E2 D2 D1 E3 D3 E4 D4
 8) 2 1 4 3  POSSIVEL    E1 E2 D2 D1 E3 E4 D4 D3
 9) 2 3 1 4  POSSIVEL    E1 E2 D2 E3 D3 D1 E4 D4
10) 2 3 4 1  POSSIVEL    E1 E2 D2 E3 D3 E4 D4 D1
11) 2 4 1 3  IMPOSSIVEL
12) 2 4 3 1  POSSIVEL    E1 E2 D2 E3 E4 D4 D3 D1
13) 3 1 2 4  IMPOSSIVEL
14) 3 1 4 2  IMPOSSIVEL
15) 3 2 1 4  POSSIVEL    E1 E2 E3 D3 D2 D1 E4 D4
16) 3 2 4 1  POSSIVEL    E1 E2 E3 D3 D2 E4 D4 D1
17) 3 4 1 2  IMPOSSIVEL
18) 3 4 2 1  POSSIVEL    E1 E2 E3 D3 E4 D4 D2 D1
19) 4 1 2 3  IMPOSSIVEL
20) 4 1 3 2  IMPOSSIVEL
21) 4 2 1 3  IMPOSSIVEL
22) 4 2 3 1  IMPOSSIVEL
23) 4 3 1 2  IMPOSSIVEL
24) 4 3 2 1  POSSIVEL    E1 E2 E3 E4 D4 D3 D2 D1

14 possiveis e 10 impossiveis (de 24)
```

Questão 3 – Árvore Binária de Busca (ABB)

O programa constrói a árvore inicial por inserções sucessivas (50, 30, 70, 20, 40, 60, 80, 10, 25, 65), executa a sequência de operações do enunciado e imprime o percurso em ordem após cada passo. Os desenhos das árvores estão no item (a). Na remoção com dois filhos o substituto é o **predecessor** (maior elemento da subárvore esquerda).

Código-fonte

```
/* QUESTAO 3 - Arvore Binaria de Busca (ABB) */
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct arv {
    int info;
    struct arv *esq;
    struct arv *dir;
} Arv;

Arv* abb_cria_no(int valor) {
    Arv *no = (Arv*) malloc(sizeof(Arv));
    no->info = valor;
    no->esq = NULL;
    no->dir = NULL;
    return no;
}

/* 1. Insercao: menores a esquerda, maiores a direita (repetidos sao ignorados) */
Arv* abb_insere(Arv *raiz, int valor) {
    if (raiz == NULL) return abb_cria_no(valor);
    if (valor < raiz->info) raiz->esq = abb_insere(raiz->esq, valor);
    else if (valor > raiz->info) raiz->dir = abb_insere(raiz->dir, valor);
    return raiz;
}

/* 2. Busca recursiva: retorna o no encontrado ou NULL */
Arv* abb_busca(Arv *raiz, int valor) {
    if (raiz == NULL || raiz->info == valor) return raiz;
    if (valor < raiz->info) return abb_busca(raiz->esq, valor);
    return abb_busca(raiz->dir, valor);
}

/* Maior elemento de uma subarvore (no mais a direita) = predecessor */
Arv* abb_maximo(Arv *raiz) {
    while (raiz->dir != NULL) raiz = raiz->dir;
    return raiz;
}

/* 3. Remocao: tres casos; com dois filhos usa o predecessor */
Arv* abb_remove(Arv *raiz, int valor) {
    if (raiz == NULL) return NULL;

    if (valor < raiz->info) {
        raiz->esq = abb_remove(raiz->esq, valor);
    } else if (valor > raiz->info) {
        raiz->dir = abb_remove(raiz->dir, valor);
    } else {
        if (raiz->esq == NULL && raiz->dir == NULL) {          /* caso 1: folha */
            free(raiz);
            return NULL;
        }
        if (raiz->esq == NULL || raiz->dir == NULL) {          /* caso 2: um filho */
            Arv *filho = (raiz->esq != NULL) ? raiz->esq : raiz->dir;
            free(raiz);
            return filho;
        }
        {                                                      /* caso 3: dois filhos */
            Arv *pred = abb_maximo(raiz->esq);
            raiz->info = pred->info;                           /* copia a chave do predecessor */
            raiz->esq = abb_remove(raiz->esq, pred->info);     /* remove o original (folha ou 1 filho) */
        }
    }
    return raiz;
}

/* 4. Altura: raiz sozinha = 0 (arvore vazia = -1) */
int abb_altura(Arv *raiz) {
    int he, hd;
    if (raiz == NULL) return -1;
    he = abb_altura(raiz->esq);
    hd = abb_altura(raiz->dir);
    return 1 + (he > hd ? he : hd);
}

/* 5. Quantidade de folhas */
int abb_folhas(Arv *raiz) {
    if (raiz == NULL) return 0;
    if (raiz->esq == NULL && raiz->dir == NULL) return 1;
    return abb_folhas(raiz->esq) + abb_folhas(raiz->dir);
```

```
}

/* 6. k-esimo menor: percurso em ordem que so conta os nos visitados (sem vetor) */
static int kesimo_aux(Arv *raiz, int k, int *cont, int *valor) {
    if (raiz == NULL) return 0;
    if (kesimo_aux(raiz->esq, k, cont, valor)) return 1;
    (*cont)++;
    if (*cont == k) {
        *valor = raiz->info;
        return 1;
    }
    return kesimo_aux(raiz->dir, k, cont, valor);
}

int abb_kesimo_menor(Arv *raiz, int k, int *valor) {
    int cont = 0;
    return kesimo_aux(raiz, k, &cont, valor);
}

/* 7. Verificacao de ABB: cada no deve estar dentro do intervalo (min, max)
 *    herdado dos ancestrais, e nao apenas ser coerente com os filhos */
static int eh_abb_aux(Arv *raiz, long long min, long long max) {
    if (raiz == NULL) return 1;
    if (raiz->info <= min || raiz->info >= max) return 0;
    return eh_abb_aux(raiz->esq, min, raiz->info) && eh_abb_aux(raiz->dir, raiz->info, max);
}

int eh_abb(Arv *raiz) {
    return eh_abb_aux(raiz, LLONG_MIN, LLONG_MAX);
}

/* 8. Percurso em ordem (ordem crescente) */
void abb_em_ordem(Arv *raiz) {
    if (raiz != NULL) {
        abb_em_ordem(raiz->esq);
        printf("%d ", raiz->info);
        abb_em_ordem(raiz->dir);
    }
}

void abb_libera(Arv *raiz) {
    if (raiz != NULL) {
        abb_libera(raiz->esq);
        abb_libera(raiz->dir);
        free(raiz);
    }
}

static void mostra(const char *titulo, Arv *raiz) {
    printf("%-28s: ", titulo);
    abb_em_ordem(raiz);
    printf("\n");
}

int main() {
    Arv *raiz = NULL;
    int i, valor;
    int inicial[] = {50, 30, 70, 20, 40, 60, 80, 10, 25, 65};

    for (i = 0; i < 10; i++) raiz = abb_insere(raiz, inicial[i]);
    mostra("Arvore inicial", raiz);

    raiz = abb_insere(raiz, 35);                      /* 1 */
    raiz = abb_insere(raiz, 75);                      /* 2 */
    mostra("Apos inserir 35 e 75", raiz);
    raiz = abb_remove(raiz, 20);                      /* 3 */
    mostra("Apos remover 20", raiz);
    raiz = abb_remove(raiz, 70);                      /* 4 */
    mostra("Apos remover 70", raiz);
    raiz = abb_remove(raiz, 50);                      /* 5 */
    mostra("Apos remover 50", raiz);
    raiz = abb_insere(raiz, 55);                      /* 6 */
    raiz = abb_insere(raiz, 15);                      /* 7 */

    printf("\nb) Percurso em ordem final : ");
    abb_em_ordem(raiz);
    printf("\nc) Altura da arvore        : %d", abb_altura(raiz));
    printf("\nd) Numero de folhas        : %d", abb_folhas(raiz));
    if (abb_kesimo_menor(raiz, 5, &valor))
        printf("\ne) 5o menor elemento       : %d", valor);
    printf("\neh_abb(arvore final)       : %d", eh_abb(raiz));
    printf("\nabb_busca(35)              : %s\n", abb_busca(raiz, 35) ? "encontrado" : "NULL");
    abb_libera(raiz);
    return 0;
}
```

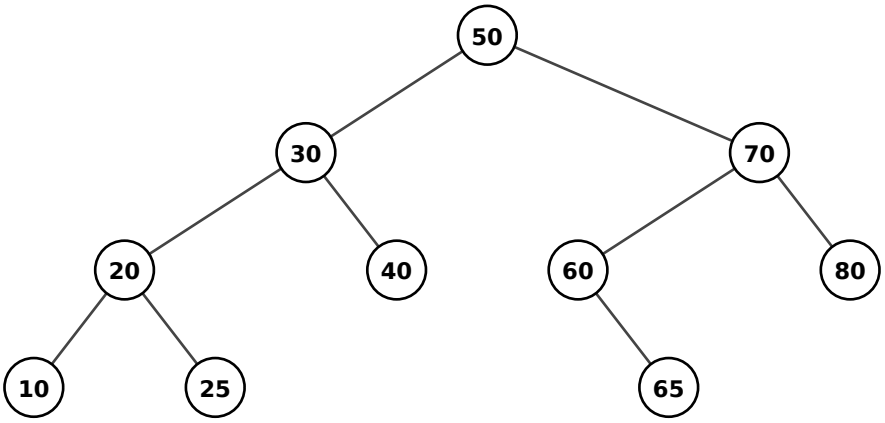
Saída da execução

```
Arvore inicial              : 10 20 25 30 40 50 60 65 70 80
Apos inserir 35 e 75        : 10 20 25 30 35 40 50 60 65 70 75 80
Apos remover 20             : 10 25 30 35 40 50 60 65 70 75 80
Apos remover 70             : 10 25 30 35 40 50 60 65 75 80
Apos remover 50             : 10 25 30 35 40 60 65 75 80
```

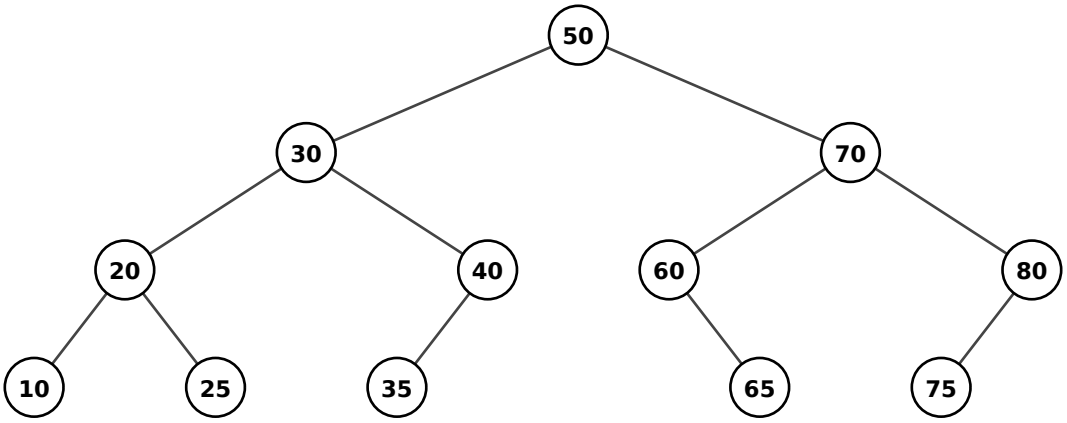
```
b) Percurso em ordem final : 10 15 25 30 35 40 55 60 65 75 80
c) Altura da arvore        : 4
d) Numero de folhas        : 4
e) 5o menor elemento       : 35
eh_abb(arvore final)       : 1
abb_busca(35)              : encontrado
```

Tarefas adicionais

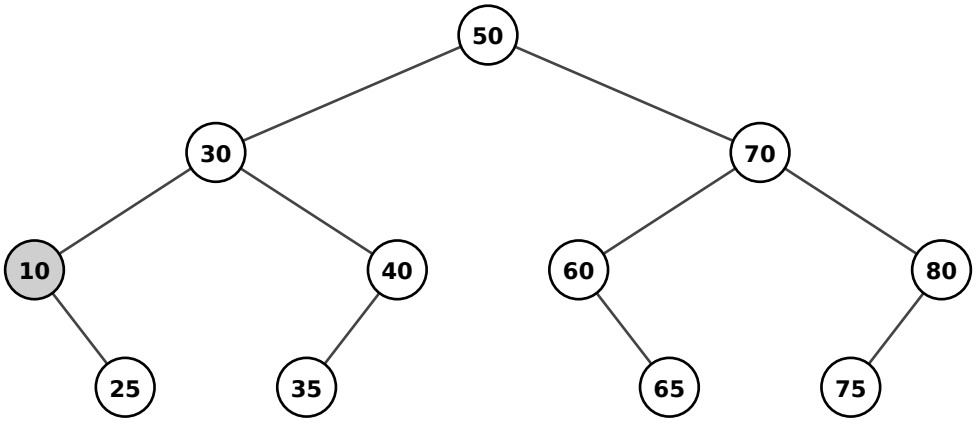
a) Árvore após cada operação (desenhos)



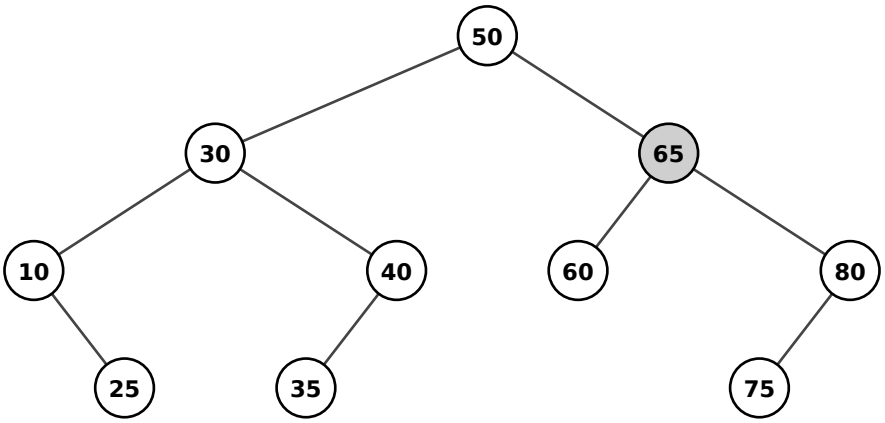
Árvore inicial (10 nós)



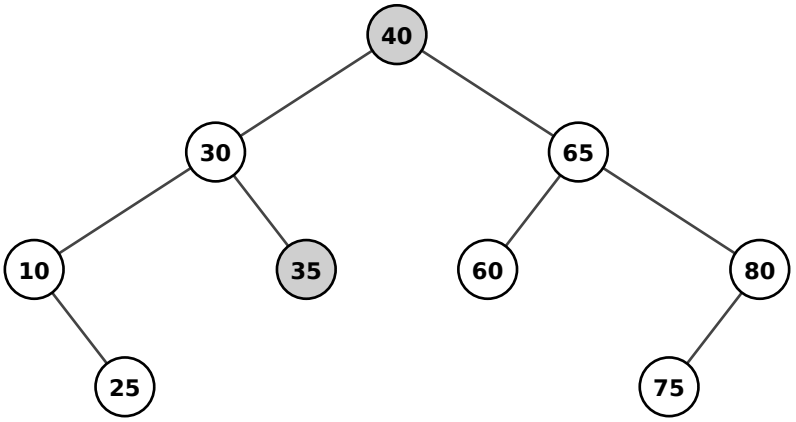
Após inserir 35 e 75 (operações 1 e 2)



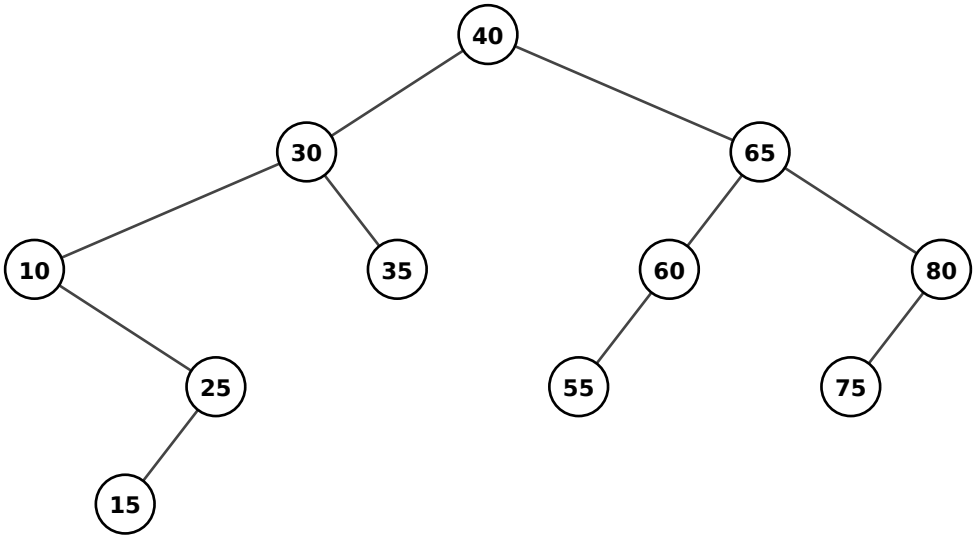
Após **remover 20** (operação 3): nó com dois filhos; o predecessor 10 ocupa o seu lugar



Após **remover 70** (operação 4): o predecessor 65 ocupa o lugar do 70



Após **remover 50** (operação 5): o predecessor 40 vira a raiz e o 35 sobe para o lugar do 40



Árvore final, após inserir 55 e 15 (operações 6 e 7)

b) Percurso em ordem ao final das operações

`10 15 25 30 35 40 55 60 65 75 80` (ordem crescente, como esperado para uma ABB).

c) Altura da árvore final

Altura = 4 (considerando a raiz com altura 0). O caminho mais longo é 40 → 30 → 10 → 25 → 15, com 4 arestas.

d) Número de folhas

4 folhas: 15, 35, 55 e 75.

e) 5º menor elemento (`abb_kesimo_menor(raiz, 5, &valor)`**)**

35. Em ordem: 10 (1º), 15 (2º), 25 (3º), 30 (4º), **35 (5º)**. A função faz um percurso em ordem que apenas conta os nós visitados (variável `cont`) e para ao chegar em *k*; não usa vetor nem outra estrutura auxiliar.

f) Como `abb_remove()` **trata a remoção do 70 e depois do 50**

Remoção do 70 (árvore antes: raiz 50; subárvore direita 70 com filhos 60 e 80; 60 tem filho direito 65; 80 tem filho esquerdo 75):

- 70 > 50: a função desce para a direita até encontrar o nó 70.
- O 70 tem **dois filhos** (60 e 80), então cai no caso 3.
- `abb_maximo(raiz->esq)` procura o maior da subárvore esquerda (a do 60): 60 → 65 (sem filho direito). O **predecessor é 65**.
- A chave 65 é copiada para o nó que continha 70 (a estrutura da árvore não muda nessa etapa).
- Chama-se `abb_remove(raiz->esq, 65)`: 65 > 60, desce à direita e encontra o 65 original, que é **folha** (caso 1) e é liberado. Resultado: o nó 65 tem filho esquerdo 60 e filho direito 80.

Remoção do 50 (a raiz; árvore antes: esquerda 30 com filhos 10 e 40; 40 tem filho esquerdo 35):

- O 50 é encontrado imediatamente e tem **dois filhos** (caso 3).
- O predecessor é o maior da subárvore esquerda: 30 → 40 (o 40 não tem filho direito). O **predecessor é 40**.
- A chave 40 é copiada para a raiz; agora é preciso remover o 40 original da subárvore esquerda.
- `abb_remove(raiz->esq, 40)`: 40 > 30, desce à direita e encontra o 40, que tem **um único filho** (35) – caso 2. O nó é liberado e o 35 sobe para o lugar dele (filho direito do 30).

Em ambos os casos o predecessor é escolhido porque é o maior valor menor que o removido: copiá-lo para o nó preserva a propriedade da ABB (tudo à esquerda continua menor, tudo à direita continua maior), e o nó original do predecessor nunca tem filho direito, portanto cai sempre no caso de folha ou de um filho.

Questão 4 – Árvore AVL

Inserções, nesta ordem: 50, 30, 70, 20, 40, 10, 25, 35, 45, 60, 80, 90, 85. O fator de balanceamento (FB) é *altura(esq) − altura(dir)*, com altura de nó nulo = −1 e de folha = 0. A função `balancear` é chamada em cada nó do caminho de volta da recursão, tanto em `inserir` quanto em `remover`: se |FB| > 1, identifica o caso (LL, RR, LR ou RL), imprime o diagnóstico e aplica as rotações. A árvore nunca é reconstruída; o balanceamento é sempre dinâmico.

a) e b) Código-fonte (estrutura do nó, altura, FB, rotações, inserir, remover)

```
/* QUESTAO 4 - Arvore AVL (insercao, remocao e rotacoes LL, RR, LR, RL) */
#include <stdio.h>
#include <stdlib.h>

/* a) Estrutura do no */
typedef struct no {
    int chave;
    struct no *esq;
    struct no *dir;
    int altura;
} No;

static int rotacoes = 0;       /* conta as rotacoes (para informar se houve alguma) */

/* b) Funcoes basicas (altura de no nulo = -1, de folha = 0) */
int altura(No *no) {
    return (no == NULL) ? -1 : no->altura;
}

static void atualizaAltura(No *no) {
    int he = altura(no->esq), hd = altura(no->dir);
    no->altura = 1 + (he > hd ? he : hd);
}

int fatorBalanceamento(No *no) {
    return (no == NULL) ? 0 : altura(no->esq) - altura(no->dir);
}

/* Rotacao simples a direita (caso LL): o filho esquerdo x sobe e y vira seu filho direito */
No* rotacaoDireita(No *y) {
    No *x = y->esq;
    y->esq = x->dir;
    x->dir = y;
    atualizaAltura(y);         /* y primeiro: agora esta abaixo de x */
    atualizaAltura(x);
    rotacoes++;
    return x;
}

/* Rotacao simples a esquerda (caso RR): o filho direito y sobe e x vira seu filho esquerdo */
No* rotacaoEsquerda(No *x) {
    No *y = x->dir;
    x->dir = y->esq;
    y->esq = x;
    atualizaAltura(x);         /* x primeiro: agora esta abaixo de y */
    atualizaAltura(y);
    rotacoes++;
    return y;
}

/* Rebalanceia um no (usado por inserir e remover) e informa o caso ocorrido */
static No* balancear(No *no) {
    int fb;
    atualizaAltura(no);
    fb = fatorBalanceamento(no);

    if (fb > 1) {                                       /* pesada a esquerda */
        if (fatorBalanceamento(no->esq) >= 0) {
            printf("  >> no %d, FB = %+d: caso LL -> rotacao direita em %d\n", no->chave, fb, no->chave);
            return rotacaoDireita(no);
        }
        printf("  >> no %d, FB = %+d: caso LR -> rotacao esquerda em %d e direita em %d\n",
               no->chave, fb, no->esq->chave, no->chave);
        no->esq = rotacaoEsquerda(no->esq);
        return rotacaoDireita(no);
    }
    if (fb < -1) {                                      /* pesada a direita */
        if (fatorBalanceamento(no->dir) <= 0) {
            printf("  >> no %d, FB = %+d: caso RR -> rotacao esquerda em %d\n", no->chave, fb, no->chave);
            return rotacaoEsquerda(no);
        }
        printf("  >> no %d, FB = %+d: caso RL -> rotacao direita em %d e esquerda em %d\n",
               no->chave, fb, no->dir->chave, no->chave);
        no->dir = rotacaoDireita(no->dir);
        return rotacaoEsquerda(no);
    }
    return no;
}
```

```c
static No* criaNo(int chave) {
    No *no = (No*) malloc(sizeof(No));
    no->chave = chave;
    no->esq = NULL;
    no->dir = NULL;
    no->altura = 0;
    return no;
}

/* Insercao: como em uma ABB, rebalanceando na volta da recursao */
No* inserir(No *raiz, int chave) {
    if (raiz == NULL) return criaNo(chave);
    if (chave < raiz->chave) raiz->esq = inserir(raiz->esq, chave);
    else if (chave > raiz->chave) raiz->dir = inserir(raiz->dir, chave);
    else return raiz;                                       /* repetida: ignora */
    return balancear(raiz);
}

/* f) Remocao: como em uma ABB (predecessor no caso de dois filhos), rebalanceando na volta */
No* remover(No *raiz, int chave) {
    if (raiz == NULL) return NULL;

    if (chave < raiz->chave) {
        raiz->esq = remover(raiz->esq, chave);
    } else if (chave > raiz->chave) {
        raiz->dir = remover(raiz->dir, chave);
    } else if (raiz->esq == NULL || raiz->dir == NULL) {  /* folha ou um filho */
        No *filho = (raiz->esq != NULL) ? raiz->esq : raiz->dir;
        free(raiz);
        return filho;
    } else {                                                /* dois filhos */
        No *pred = raiz->esq;
        while (pred->dir != NULL) pred = pred->dir;       /* maior da subarvore esquerda */
        raiz->chave = pred->chave;
        raiz->esq = remover(raiz->esq, pred->chave);
    }
    return balancear(raiz);
}

/* e) Percursos */
void preOrdem(No *raiz) {
    if (raiz != NULL) { printf("%d ", raiz->chave); preOrdem(raiz->esq); preOrdem(raiz->dir); }
}
void emOrdem(No *raiz) {
    if (raiz != NULL) { emOrdem(raiz->esq); printf("%d ", raiz->chave); emOrdem(raiz->dir); }
}
void posOrdem(No *raiz) {
    if (raiz != NULL) { posOrdem(raiz->esq); posOrdem(raiz->dir); printf("%d ", raiz->chave); }
}

/* Altura e fator de balanceamento de cada no, em ordem crescente de chave */
static void listaFB(No *raiz) {
    if (raiz != NULL) {
        listaFB(raiz->esq);
        printf("   no %3d : altura = %d | FB = %+d\n", raiz->chave, raiz->altura, fatorBalanceamento(raiz));
        listaFB(raiz->dir);
    }
}

static void liberaArvore(No *raiz) {
    if (raiz != NULL) {
        liberaArvore(raiz->esq);
        liberaArvore(raiz->dir);
        free(raiz);
    }
}

int main() {
    No *raiz = NULL;
    int i;
    int entrada[] = {50, 30, 70, 20, 40, 10, 25, 35, 45, 60, 80, 90, 85};
    int remocoes[] = {30, 70, 50};

    printf("=== INSERCOES ===\n");
    for (i = 0; i < 13; i++) {
        printf("Inserindo %d\n", entrada[i]);
        raiz = inserir(raiz, entrada[i]);
    }

    printf("\n=== ARVORE FINAL ===\n");
    printf("Pre-ordem : "); preOrdem(raiz);  printf("\n");
    printf("Em ordem  : "); emOrdem(raiz);   printf("\n");
    printf("Pos-ordem : "); posOrdem(raiz);  printf("\n");
    printf("Altura da arvore: %d\n", altura(raiz));
    listaFB(raiz);

    printf("\n=== REMOCOES ===\n");
    for (i = 0; i < 3; i++) {
        rotacoes = 0;
        printf("Removendo %d\n", remocoes[i]);
        raiz = remover(raiz, remocoes[i]);
        if (rotacoes == 0) printf("  Rotacao necessaria? NAO\n");
        printf("  Em ordem: "); emOrdem(raiz); printf("\n");
        printf("  Altura da arvore: %d\n", altura(raiz));
```

```
        listaFB(raiz);
        printf("\n");
    }
    liberaArvore(raiz);
    return 0;
}
```

Saída da execução

```
=== INSERCOES ===
Inserindo 50
Inserindo 30
Inserindo 70
Inserindo 20
Inserindo 40
Inserindo 10
  >> no 50, FB = +2: caso LL -> rotacao direita em 50
Inserindo 25
Inserindo 35
Inserindo 45
Inserindo 60
Inserindo 80
Inserindo 90
  >> no 30, FB = -2: caso RR -> rotacao esquerda em 30
Inserindo 85
  >> no 80, FB = -2: caso RL -> rotacao direita em 90 e esquerda em 80

=== ARVORE FINAL ===
Pre-ordem : 50 30 20 10 25 40 35 45 70 60 85 80 90
Em ordem  : 10 20 25 30 35 40 45 50 60 70 80 85 90
Pos-ordem : 10 25 20 35 45 40 30 60 80 90 85 70 50
Altura da arvore: 3
   no  10 : altura = 0 | FB = +0
   no  20 : altura = 1 | FB = +0
   no  25 : altura = 0 | FB = +0
   no  30 : altura = 2 | FB = +0
   no  35 : altura = 0 | FB = +0
   no  40 : altura = 1 | FB = +0
   no  45 : altura = 0 | FB = +0
   no  50 : altura = 3 | FB = +0
   no  60 : altura = 0 | FB = +0
   no  70 : altura = 2 | FB = -1
   no  80 : altura = 0 | FB = +0
   no  85 : altura = 1 | FB = +0
   no  90 : altura = 0 | FB = +0

=== REMOCOES ===
Removendo 30
  Rotacao necessaria? NAO
  Em ordem: 10 20 25 35 40 45 50 60 70 80 85 90
  Altura da arvore: 3
   no  10 : altura = 0 | FB = +0
   no  20 : altura = 1 | FB = +1
   no  25 : altura = 2 | FB = +0
   no  35 : altura = 0 | FB = +0
   no  40 : altura = 1 | FB = +0
   no  45 : altura = 0 | FB = +0
   no  50 : altura = 3 | FB = +0
   no  60 : altura = 0 | FB = +0
   no  70 : altura = 2 | FB = -1
   no  80 : altura = 0 | FB = +0
   no  85 : altura = 1 | FB = +0
   no  90 : altura = 0 | FB = +0

Removendo 70
  >> no 60, FB = -2: caso RR -> rotacao esquerda em 60
  Em ordem: 10 20 25 35 40 45 50 60 80 85 90
  Altura da arvore: 3
   no  10 : altura = 0 | FB = +0
   no  20 : altura = 1 | FB = +1
   no  25 : altura = 2 | FB = +0
   no  35 : altura = 0 | FB = +0
   no  40 : altura = 1 | FB = +0
   no  45 : altura = 0 | FB = +0
   no  50 : altura = 3 | FB = +0
   no  60 : altura = 1 | FB = -1
   no  80 : altura = 0 | FB = +0
   no  85 : altura = 2 | FB = +1
   no  90 : altura = 0 | FB = +0

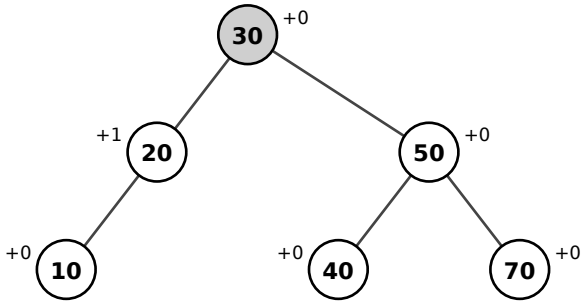
Removendo 50
  Rotacao necessaria? NAO
  Em ordem: 10 20 25 35 40 45 60 80 85 90
  Altura da arvore: 3
   no  10 : altura = 0 | FB = +0
   no  20 : altura = 1 | FB = +1
   no  25 : altura = 2 | FB = +0
   no  35 : altura = 0 | FB = +0
   no  40 : altura = 1 | FB = +1
   no  45 : altura = 3 | FB = +0
   no  60 : altura = 1 | FB = -1
   no  80 : altura = 0 | FB = +0
   no  85 : altura = 2 | FB = +1
   no  90 : altura = 0 | FB = +0
```

c) Desbalanceamentos durante as inserções

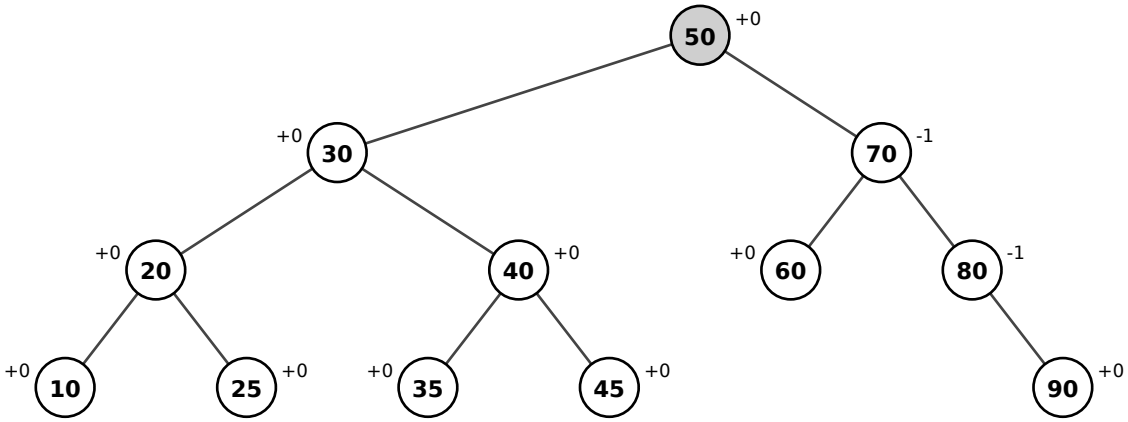
Em todas as demais inserções a árvore permaneceu balanceada. Ocorreram exatamente três desbalanceamentos:

Inserção	Nó desbalanceado	FB	Caso	Rotação necessária
10	50	+2 (filho esq. 30 com FB +1)	LL	simples à direita em 50
90	30	−2 (filho dir. 50 com FB −1)	RR	simples à esquerda em 30
85	80	−2 (filho dir. 90 com FB +1)	RL	dupla: direita em 90, depois esquerda em 80

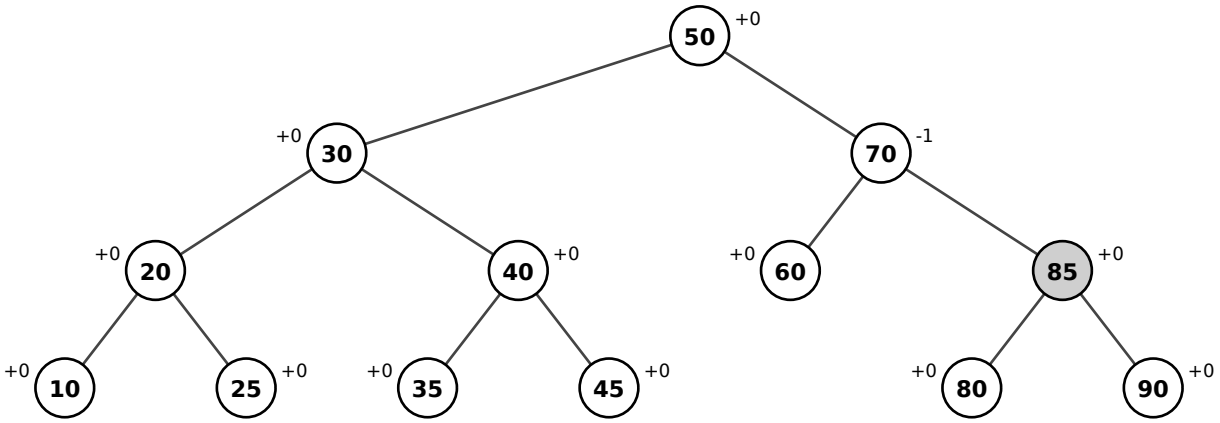
d) Árvore após cada inserção que provocou rotação (FB de cada nó ao lado; nó destacado = nova raiz da subárvore rotacionada)



Após inserir 10: rotação simples à direita em 50 (caso LL). O 30 assume a raiz.



Após inserir 90: rotação simples à esquerda em 30 (caso RR). O 50 assume a raiz.



Após inserir 85: rotação dupla direita-esquerda (caso RL). O 85 assume o lugar do 80.

e) Resultados ao final da construção

Percurso	Resultado
Pré-ordem	`50 30 20 10 25 40 35 45 70 60 85 80 90`
Em ordem	`10 20 25 30 35 40 45 50 60 70 80 85 90`
Pós-ordem	`10 25 20 35 45 40 30 60 80 90 85 70 50`
Altura da árvore	**3**

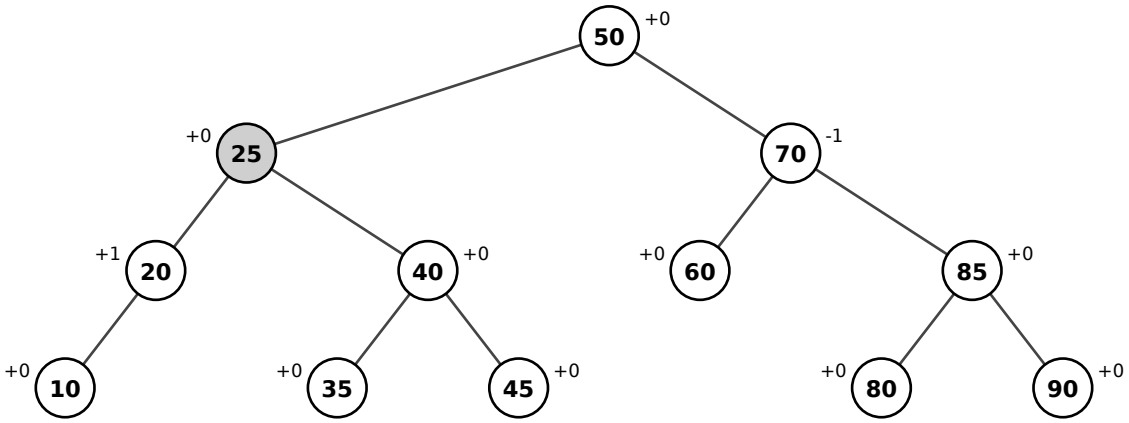
Nó	10	20	25	30	35	40	45	50	60	70	80	85	90
Altura	0	1	0	2	0	1	0	3	0	2	0	1	0
FB	+0	+0	+0	+0	+0	+0	+0	+0	+0	-1	+0	+0	+0

Todos os nós têm FB = 0, exceto o nó 70, que tem FB = −1 (altura 0 à esquerda, altura 1 à direita). Portanto, todos os |FB| ≤ 1 e a árvore é uma AVL válida.

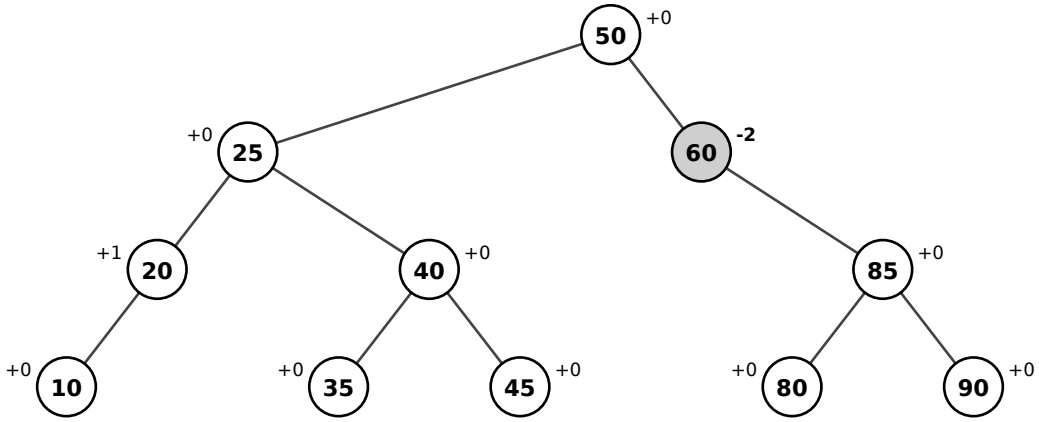
f) Remoção de 30, 70 e 50 (com predecessor)

A função `remover` aplica as mesmas regras da ABB (folha, um filho, dois filhos com o predecessor = maior da subárvore esquerda) e, ao voltar da recursão, chama `balancear` em cada ancestral. Resumo:

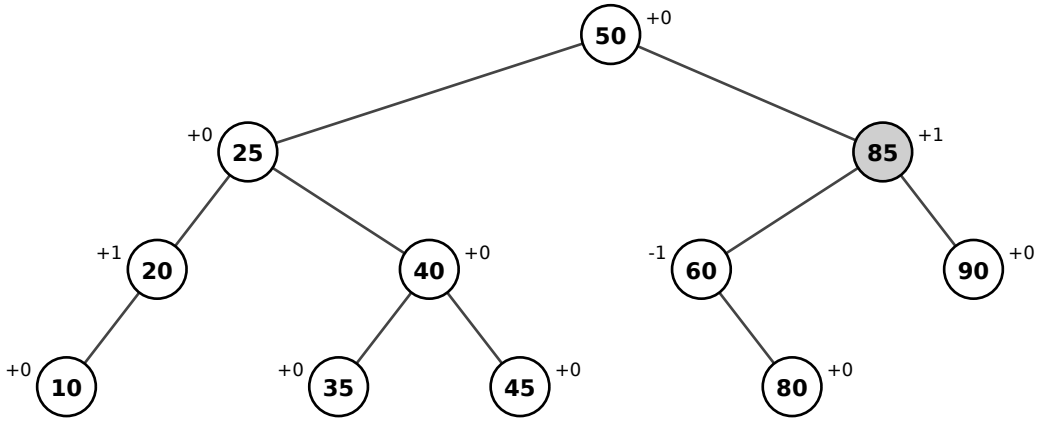
Remoção	Predecessor	Rotação?	Detalhe
30	25	Não	25 substitui o 30; o 20 fica só com o filho 10 (FB +1). Nenhum nó chega a |FB| = 2.
70	60	**Sim – RR**, rotação simples à esquerda no nó 60	O 60 assume o lugar do 70 e é removido da esquerda; o nó (agora com chave 60) fica sem filho esquerdo e com a subárvore do 85 à direita: FB = −2. O filho direito (85) tem FB = 0, tratado como RR. Após a rotação o **85** sobe.
50	45	Não	45 substitui a raiz; o 40 fica só com o filho 35 (FB +1). Raiz com FB 0.



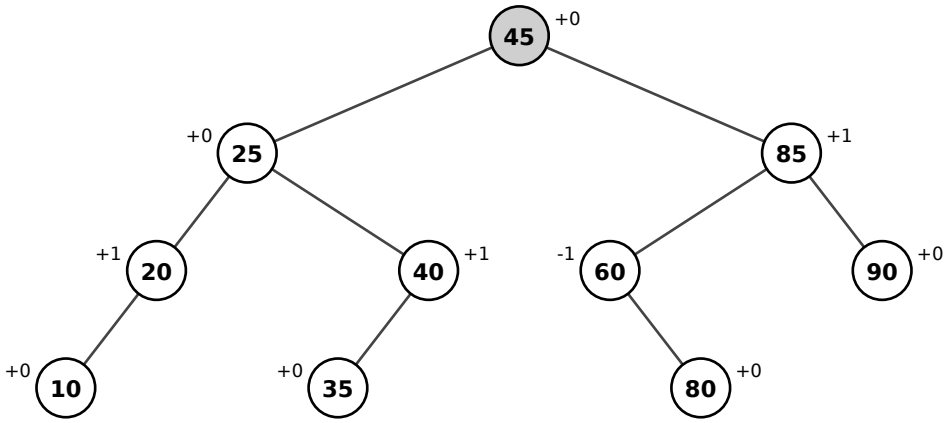
Após **remover 30**: o predecessor 25 ocupa o lugar do 30. Sem rotação.



Remover 70, **antes da rotação**: o predecessor 60 ocupa o lugar do 70 e o nó fica com FB = −2 (caso RR).



Após **remover 70**: rotação simples à esquerda no nó 60 (caso RR); o 85 sobe e o 60 passa a ser seu filho esquerdo.



Após **remover 50**: o predecessor 45 vira a raiz. Sem rotação.

Fatores de balanceamento (formato chave:FB, em ordem crescente) e altura após cada remoção

Remoção	FB de cada nó	Em ordem	Altura
30	`10:+0 20:+1 25:+0 35:+0 40:+0 45:+0 50:+0 60:+0 70:-1 80:+0 85:+0 90:+0`	`10 20 25 35 40 45 50 60 70 80 85 90`	3
70	`10:+0 20:+1 25:+0 35:+0 40:+0 45:+0 50:+0 60:-1 80:+0 85:+1 90:+0`	`10 20 25 35 40 45 50 60 80 85 90`	3
50	`10:+0 20:+1 25:+0 35:+0 40:+1 45:+0 60:-1 80:+0 85:+1 90:+0`	`10 20 25 35 40 45 60 80 85 90`	3

Após as três remoções todos os nós têm |FB| ≤ 1.

Observação sobre predecessor × sucessor. O resultado depende dessa escolha. Se fosse usado o sucessor (menor elemento da subárvore direita), as remoções 30, 70 e 50 usariam 35, 80 e 60, respectivamente; nesse caso só a remoção do 50 provoca rotação (RR no nó 80) e a raiz final é o 60. Neste trabalho foi usado o **predecessor**, como pedido na Questão 3.

Desafio adicional – por que a AVL é O(log n) e a ABB pode ser O(n)

O tempo de busca, inserção e remoção em árvores de busca é proporcional à **altura** da árvore (o caminho mais longo da raiz até uma folha).

ABB convencional. A forma da árvore depende da ordem de inserção. Se os dados chegam ordenados (por exemplo 10, 20, 30, 40, 50), cada novo valor vai para a direita do anterior e a árvore degenera numa lista encadeada: com n nós a altura é $n - 1$ e achar ou inserir o último elemento exige percorrer os n nós. Esse pior caso é $O(n)$.

AVL. Exige que o FB de todo nó esteja em $\{-1, 0, +1\}$. Sempre que uma inserção ou remoção viola essa regra, uma rotação (LL, RR, LR ou RL) restaura o equilíbrio em tempo O(1), e apenas nós do caminho da raiz até o ponto da alteração precisam ser verificados.

Por que a altura é logarítmica. Seja N(h) o *menor* número de nós de uma AVL de altura h. Para ter altura h, a raiz precisa ter um filho de altura h − 1 e o outro de altura pelo menos h − 2, logo

$$N(h) = N(h-1) + N(h-2) + 1, \text{ com } N(0) = 1 \text{ e } N(1) = 2.$$

h	0	1	2	3	4	5	6
N(h) mínimo	1	2	4	7	12	20	33

Essa recorrência cresce como a de Fibonacci, ou seja, exponencialmente (aproximadamente φ^h, com $\varphi \approx 1{,}618$). Assim, uma AVL com n nós tem altura $h < 1{,}44 \cdot \log_2(n + 2)$, isto é, $h = O(\log n)$. Como busca, inserção e remoção percorrem um único caminho e fazem trabalho constante por nível, todas custam **O(log n)** no pior caso. Para $n = 1.000.000$, uma ABB degenerada pode exigir cerca de 1.000.000 de passos, enquanto a AVL nunca passa de aproximadamente 29.

Repositório com os códigos completos: https://github.com/aliceb404/Estrutura-de-dados-II/tree/main

Discentes: Alice Fernanda Silva Barros, João Emannuel Neves e João Victor Lisboa.